

WECA R & Quarto Training

From Excel to Code-First Analysis

WECA Analysis Team

2026-02-17

Table of contents

1	WECA R & Quarto Training	5
1.1	Welcome	5
1.2	Course Structure	5
1.2.1	Session Overview	5
1.3	Learning Outcomes	6
1.4	Course Philosophy	6
1.5	Getting Started	6
1.5.1	Prerequisites	6
1.5.2	Course Materials	6
1.6	Navigation	6
I	Course Overview	8
2	Course Plan Overview	9
2.1	Course Design Philosophy	9
2.2	Learning Outcomes	9
2.3	Session Breakdown	9
2.4	Pre-Session Setup (Instructor)	10
2.5	Teaching Philosophy	10
2.6	Assessment Strategy	10
2.7	Success Metrics	11
2.8	Post-Course Support	11
3	Teaching Pedagogy	12
3.1	Evidence-Based Approaches	12
3.2	Adopt a “Tidyverse-First” Approach	12
3.3	Prioritise Data Visualisation	12
3.4	Centre Learning on Real-World Case Studies	13
3.5	Integrate Literate Programming and Reproducibility	13
3.6	Foster Active Learning and “Statistical Thinking”	13
3.7	Learning Science Principles Applied	14
3.7.1	Cognitive Load Management	14
3.7.2	Active Learning	14
3.7.3	Scaffolding	14
3.7.4	Social Learning	14
3.7.5	Metacognition	14

3.7.6	Transfer of Learning	14
3.8	References	15

II Training Sessions 16

4	Session 1: The Whole Game	17
4.1	Learning Outcomes	17
4.2	Session Structure	17
4.2.1	Part 1: Welcome & Context (30 min)	17
4.2.2	Part 2: The Whole Game Walkthrough (90 min)	18
4.2.3	Part 3: Understanding the Pieces (45 min)	18
4.2.4	Part 4: Your First Modification (60 min)	19
4.2.5	Part 5: Wrap-up & Homework (30 min)	19
5	Session 2: R Fundamentals	21
5.1	Learning Outcomes	21
5.2	Session Structure	21
5.2.1	Part 1: Homework Review & Setup (30 min)	21
5.2.2	Part 2: R Basics - Data Structures (45 min)	22
5.2.3	Part 3: The Tidyverse Way - Verbs and Pipes (60 min)	22
5.2.4	Part 4: ggplot2 - Grammar of Graphics (60 min)	23
5.2.5	Part 5: Debugging R Errors (30 min)	24
5.2.6	Part 6: Wrap-up & Homework (15 min)	24
6	Session 3: Data Wrangling	25
6.1	Learning Outcomes	25
6.2	Session Structure	25
6.2.1	Part 1: Homework Review & Setup (30 min)	25
6.2.2	Part 2: Importing Data (45 min)	26
6.2.3	Part 3: Data Cleaning Essentials (60 min)	27
6.2.4	Part 4: Reshaping Data (60 min)	27
6.2.5	Part 5: Joining Datasets (30 min)	28
6.2.6	Common Pitfalls	28
6.2.7	Part 6: Real Data Workshop (45 min)	29
6.2.8	Part 7: Wrap-up & Homework (15 min)	29
7	Session 4: Quarto Mastery	30
7.1	Learning Outcomes	30
7.2	Session Structure	30
7.2.1	Part 1: Homework Review (30 min)	30
7.2.2	Part 2: Markdown Essentials (45 min)	30
7.2.3	Part 3: Quarto Code Chunks (60 min)	31
7.2.4	Part 4: Tables in Quarto (45 min)	32
7.2.5	Part 5: Document Structure & Cross-References (45 min)	33
7.2.6	Common Pitfalls	33
7.2.7	Part 6: The Freeze Workflow for Collaboration (30 min)	34
7.2.8	Part 7: Rendering Options (15 min)	34
7.2.9	Part 8: Practical Exercise (45 min)	34
7.2.10	Part 9: Wrap-up & Homework (15 min)	35

8	Session 5: Git Collaboration	36
8.1	Learning Outcomes	36
8.2	Session Structure	36
8.2.1	Part 1: Homework Review & Version Control Intro (45 min)	36
8.2.2	Part 2: Git Basics in Git Bash (60 min)	37
8.2.3	Part 3: Branching Workflow (60 min)	38
8.2.4	Part 4: Pull Requests on GitHub (60 min)	38
8.2.5	Part 5: Merging and Updating (30 min)	39
8.2.6	Part 6: Merge Conflicts (Basic) (30 min)	40
8.2.7	Part 7: WECA Project Git Workflow (20 min)	40
8.2.8	Part 8: Final Practise - Full Workflow (45 min)	41
8.2.9	Part 9: Course Wrap-up (30 min)	41
III	Resources	42
9	Practice Datasets	43
9.1	Example Datasets	43
9.1.1	Bus Ridership	43
9.1.2	Area Employment	43
9.2	Real Indicator Data	44
9.3	Data Cleaning Notes	44
9.4	Dataset Documentation Template	44
10	Setup Guide	46
10.1	Pre-Session Setup Checklist	46
10.2	Software Installation	46
10.2.1	Required Software	46
10.3	Project Setup	47
10.3.1	Clone the Indicators Repository	47
10.3.2	Install R Packages	47
10.3.3	Install Python Dependencies	47
10.3.4	Install Pre-Commit Hooks	47
10.4	Verify Setup	48
10.4.1	Check R Environment	48
10.4.2	Check Quarto Rendering	48
10.4.3	Check Git Configuration	48
10.4.4	Test Full Workflow	48
10.5	Prepare Practice Data	48
10.5.1	Example Datasets	48
10.5.2	Create Practice Branches	49
10.6	Troubleshooting Setup	49
10.7	Post-Setup Verification	49
10.8	Support Resources	49
11	Troubleshooting Guide	51
11.1	Setup Issues	51
11.1.1	“Quarto not found”	51
11.1.2	“Python package installation fails”	51
11.1.3	“renv restore hangs”	51

11.2	Common Issues and Solutions	52
11.2.1	R Issues	52
11.2.2	Git Issues	53
11.2.3	Quarto Issues	54
11.2.4	Positron Issues	55
11.2.5	Data Issues	56
11.2.6	Performance Issues	57
11.3	Getting Help	57
11.3.1	Self-Help Resources	57
11.3.2	Asking for Help	58
11.3.3	Office Hours	58
11.3.4	Team Support	58

Chapter 1

WECA R & Quarto Training

From Excel to Code-First Analysis

1.1 Welcome

This training course helps WECA analysts transition from Excel/PowerPoint workflows to reproducible, code-first analysis using R, Quarto, and Git.

Goal: Enable each analyst to independently create indicator chapters using R, Quarto, and Git.

1.2 Course Structure

- **Duration:** 5 sessions $\times$ 4 hours (20 hours total)
- **Format:** In-person, consecutive weeks
- **Audience:** 5 analysts building indicator chapters for the regional priorities report

1.2.1 Session Overview

Session	Focus	Key Milestone
1	The Whole Game	Render a complete indicator from template
2	R Fundamentals	Transform and visualise real data
3	Data Wrangling	Clean and prepare messy datasets
4	Quarto Mastery	Author professional multi-section reports
5	Git Collaboration	Create, review, and merge pull requests

1.3 Learning Outcomes

By the end of this course, you will be able to:

- Navigate the command line (Git Bash) for basic file operations
- Use Positron IDE to write and execute R code
- Load, transform, and analyse data using tidyverse packages
- Create publication-quality visualisations with ggplot2
- Author reproducible reports combining code, narrative, and outputs using Quarto
- Use Git for version control and GitHub for collaboration
- Follow the WECA indicators project workflow independently

1.4 Course Philosophy

This course follows evidence-based pedagogical principles:

1. **“Whole Game First”** - Session 1 shows the complete workflow before diving into details
2. **Tidyverse-First** - Human-centred R syntax (pipes, verbs) from the start
3. **Visual Feedback Early** - Charts in Session 1 for immediate gratification
4. **Real-World Application** - Build actual WECA indicators, not toy examples
5. **Literate Programming** - Code + narrative from day one via Quarto
6. **Active Learning** - Live coding, immediate practise, pair programming options
7. **Low Friction** - Pre-installed software, standardised helper functions

1.5 Getting Started

1.5.1 Prerequisites

Before Session 1, ensure:

1. Software installed (see [Setup Guide](#))
2. Access to the indicators project repository
3. Basic familiarity with Excel and data analysis concepts

1.5.2 Course Materials

Navigate the book using the sidebar:

- **Overview** - Course design and pedagogy
- **Sessions 1-5** - Detailed lesson plans with exercises
- **Resources** - Datasets, setup guides, troubleshooting

1.6 Navigation

Use the sidebar to access:

- Individual session materials

- Practise datasets and examples
 - Reference guides and troubleshooting
 - Additional learning resources
-

Part I

Course Overview

Chapter 2

Course Plan Overview

2.1 Course Design Philosophy

This course follows evidence-based pedagogical principles:

1. **“Whole Game First”** - Session 1 shows the complete workflow before diving into details
2. **Tidyverse-First** - Human-centered R syntax (pipes, verbs) from the start
3. **Visual Feedback Early** - Charts in Session 1 for immediate gratification
4. **Real-World Application** - Build actual WECA indicators, not toy examples
5. **Literate Programming** - Code + narrative from day one via Quarto
6. **Active Learning** - Live coding, immediate practice, pair programming options
7. **Low Friction** - Pre-installed software, standardized helper functions

2.2 Learning Outcomes

By the end of this course, analysts will be able to:

- Navigate the command line (Git Bash) for basic file operations
- Use Positron IDE to write and execute R code
- Load, transform, and analyse data using tidyverse packages
- Create publication-quality visualisations with ggplot2
- Author reproducible reports combining code, narrative, and outputs using Quarto
- Use Git for version control and GitHub for collaboration
- Follow the WECA indicators project workflow independently

2.3 Session Breakdown

Session	Focus	Key Milestone	Duration
1	The Whole Game	Render a complete indicator from template	4 hours
2	R Fundamentals	Transform and visualise real data	4 hours
3	Data Wrangling	Clean and prepare messy datasets	4 hours
4	Quarto Mastery	Author professional multi-section reports	4 hours
5	Git Collaboration	Create, review, and merge pull requests	4 hours

2.4 Pre-Session Setup (Instructor)

Week Before Session 1:

1. Install software on all machines:
 - Positron, R (4.3), Python (3.10), Quarto (1.4), Git, uv
2. Clone `indicators` repository to `~/projects/` on each machine
3. Run setup scripts:
 - `./scripts/install-hooks.sh` (pre-commit hooks)
 - `renv::restore()` in R (packages)
 - `uv sync` in terminal (Python environment)
4. Prepare 5 example datasets (CSV files) in `data/examples/`:
 - Simple, clean data (4-6 columns, 20-50 rows)
 - Topics relevant to each priority area (transport, economy, skills, etc.)
 - Variables ready for plotting (year, value, category)
5. Create practice branches for each analyst: `firstname-practice`

Data complexity progression:

- Sessions 1-2: Clean CSVs (consistent column names, no missing values, correct types, long format)
- Session 3: Messier data (mixed case column names, missing values, wide format needing pivot, multiple files needing joins)
- Sessions 4-5: Learners use their own real indicator data (full complexity)

2.5 Teaching Philosophy

This course applies evidence-based learning science principles detailed in [Teaching Pedagogy](#), including cognitive load management, active learning, scaffolding, and social learning.

2.6 Assessment Strategy

Formative Assessment (During Sessions):

- Live coding participation
- Pair programming discussions

- Homework completion
- Questions asked (indicator of engagement)

Summative Assessment (End of Course):

Each analyst can independently:

1. Import a CSV dataset
2. Clean and transform the data
3. Create a chart with WECA branding
4. Author a complete indicator section in Quarto
5. Render the chapter without errors
6. Create a pull request following the workflow

2.7 Success Metrics

Short-term (End of Course):

- 100% of analysts complete at least one indicator independently
- 80%+ create PRs following the workflow
- 100% can render a chapter without instructor help

Medium-term (1 month post-course):

- Each analyst has contributed 3+ indicators
- PRs are created without prompting
- Analysts help each other in team chat

Long-term (3 months post-course):

- Indicators project workflow is self-sustaining
- Analysts propose and implement workflow improvements
- Code quality improves (evidenced by PR reviews)

2.8 Post-Course Support

- Office hours (1 hour/week for 4 weeks)
- Dedicated team chat channel
- Pair programming sessions available
- Instructor code reviews for first 5 PRs

Chapter 3

Teaching Pedagogy

3.1 Evidence-Based Approaches

This course is built on evidence-based pedagogical principles for teaching R and data science effectively.

3.2 Adopt a “Tidyverse-First” Approach

The “tidyverse” ecosystem (including packages like `ggplot2`, `dplyr`, `tidyr`, and `readr`) is designed to be “human-centered,” prioritizing the cognitive needs of the analyst over the computer’s convenience.

Consistency and Readability: The tidyverse shares a common high-level design philosophy and low-level grammar. This means that once a learner masters one package, that knowledge transfers easily to others.

Human-Readable Code: The syntax uses intuitive verbs (e.g., `filter`, `mutate`, `select`) and the pipe operator (`|>` or `%>%`) to structure code as a linear sequence of actions, making it easier to read and write. This readability helps learners focus on the analysis logic rather than obscure programming syntax.

“Batteries Included”: Installing the single tidyverse meta-package provides a comprehensive toolkit for the core data science tasks—`import`, `tidy`, `transform`, `visualise`, and `program`—reducing the friction of managing dependencies.

3.3 Prioritise Data Visualisation

Visual feedback is a powerful motivator for learners.

Start with Visualisation: Introducing `ggplot2` early allows learners to create complex, compelling graphics with relatively little code. This provides immediate gratification and helps learners spot patterns or errors in their data quickly.

Layered Complexity: `ggplot2` implements a “grammar of graphics,” allowing learners to build plots layer-by-layer. They can start simple and progressively

add complexity (e.g., mapping additional variables to colours or shapes, or adding smooth lines) as their understanding grows.

3.4 Centre Learning on Real-World Case Studies

Traditional statistics courses often focus on theoretical properties using clean, toy datasets. Effective data science education brings applications to the forefront.

Problem-First Pedagogy: Organise learning around diverse case studies where the primary goal is answering a specific subject-matter question (e.g., “Is there gender bias in grant funding?” or “Who will win the election?”).

Real, Messy Data: Use real-world data rather than pre-cleaned examples. This exposes learners to the reality of “data wrangling” and the necessity of cleaning and structuring data before analysis.

The “Whole Game”: Structure exercises to mimic the full lifecycle of a data scientist’s work: formulating a question, gathering data, cleaning it, analyzing it, and communicating results. This teaches students to “connect” subject matter to statistical frameworks.

3.5 Integrate Literate Programming and Reproducibility

Learners gain confidence when they can document their thought process alongside their code.

Literate Programming: Tools like Quarto and R Markdown allow users to interleave narrative text, code, and output (plots/tables) in a single document. This supports rigorous thinking by encouraging learners to record why they performed an analysis, not just how.

Reproducible Workflows: Teaching reproducibility early—using R Projects, relative paths, and version control (Git/GitHub)—helps learners manage their environment and ensures their work can be verified and built upon.

Automated Reporting: Using tools like Quarto avoids the error-prone “copy-paste” cycle of moving results into Word or PowerPoint. This establishes a robust workflow where updating data or code automatically updates the final report.

3.6 Foster Active Learning and “Statistical Thinking”

Live Coding and Iteration: Instructors should write code live (using literate programming tools) to demonstrate the iterative process of analysis, including how to troubleshoot errors in real-time.

Three Key Skills: Effective instruction should aim to develop three core skills:

1. **Creating** - Formulating questions and data

2. **Connecting** - Linking questions to appropriate data and methods
3. **Computing** - Using tools like R to execute the analysis

Minimizing Notation: When introducing statistical concepts, relying on computational approaches (like simulation and visualisation) rather than heavy mathematical notation can make abstract concepts more accessible.

3.7 Learning Science Principles Applied

3.7.1 Cognitive Load Management

- “Whole Game” first reduces extraneous load (see the forest before the trees)
- Consistent patterns (load $\rightarrow$ transform $\rightarrow$ visualise) reduce cognitive overhead
- Helper functions reduce syntax complexity
- Spaced practice via homework spreads learning over time

3.7.2 Active Learning

- Live coding with immediate practice (not passive lectures)
- Minimal lecture time ($< 30\%$ of each session)
- Real-world application from Session 1

3.7.3 Scaffolding

- Pre-installed software eliminates setup friction
- Helper functions abstract complexity initially
- Example datasets are clean initially, messier in later sessions
- Gradual release: instructor demo $\rightarrow$ guided practice $\rightarrow$ independent work

3.7.4 Social Learning

- Pair programming options
- Peer code review in Session 5
- Collaborative project (shared indicators repository)

3.7.5 Metacognition

- Reflection time in each session wrap-up
- Debugging strategy explicitly taught
- Error messages explained (not feared)

3.7.6 Transfer of Learning

- All practice uses real WECA data/project structure
- No toy examples after Session 1
- Authentic task design (they build actual deliverables)

3.8 References

This pedagogy is informed by:

- [R for Data Science](#)
- [Modern Statistics for Modern Biology - Holmes & Huber](#)
- [Tidy design principles](#)

Part II

Training Sessions

Chapter 4

Session 1: The Whole Game

Experience the complete workflow

- **Duration:** 4 hours
- **Goal:** Experience the complete workflow - from data to published report - to understand *why* we're learning each tool.

4.1 Learning Outcomes

- Understand the “code-first” analysis paradigm vs. Excel workflows
- Navigate the indicators project structure
- Execute an R code chunk in Quarto
- Render a chapter to HTML
- See the role of Git in version control (conceptual)

4.2 Session Structure

4.2.1 Part 1: Welcome & Context (30 min)

Why are we here?

- The limitations of Excel/PowerPoint workflows (manual updates, copy-paste errors, unclear provenance)
- Benefits of code-first analysis: reproducibility, automation, transparency, collaboration
- Tour of the final product: rendered indicators book

The tools we'll learn:

- **Git Bash:** Command-line interface (the “steering wheel”)
- **Positron:** Integrated development environment (the “dashboard”)
- **R:** Statistical programming language (the “engine”)
- **Quarto:** Literate programming system (the “publishing press”)

- **Git/GitHub:** Version control (the “time machine + collaboration hub”)

Analogy: If analysis were cooking:

- Excel is a microwave meal kit
- Code-first is a professional kitchen with recipes you can modify, share, and improve

4.2.2 Part 2: The Whole Game Walkthrough (90 min)

Live Demonstration (30 min) - Instructor creates an indicator from scratch:

1. Open Positron
2. Navigate to `chapters/02-transport/index.qmd`
3. Write a simple R code chunk:

```
library(tidyverse)
source(here::here("scripts", "R", "theme_weca.R"))

# Load example data
bus_data <- read_csv("data/examples/bus_ridership.csv")

# Create visualisation
ggplot(bus_data, aes(x = year, y = ridership)) +
  geom_line(colour = get_weca_color("forest_green"), linewidth = 1.2) +
  geom_point(colour = get_weca_color("forest_green"), size = 3) +
  theme_weca() +
  labs(title = "Annual Bus Ridership",
       x = "Year",
       y = "Ridership (millions)")
```

4. Execute chunk (Ctrl+Shift+Enter)
5. Render chapter (Quarto: Render button)
6. View output in browser

Guided Practise (60 min) - Analysts follow along:

1. Open Positron (show how to launch from Start menu)
2. Open project: File > Open Folder > C:\Users\your.name\projects\weca_analysis_and_evaluation
3. Open their assigned example chapter (pre-created templates)
4. Execute the setup chunk (walk through line-by-line what it does)
5. Modify the plot title and colours
6. Re-render and view changes

Common Issues Troubleshooting (built into this time):

- Package not found → `renv::restore()`
- File path errors → Emphasise `here::here()`
- Rendering fails → Check R console for errors

BREAK (15 min)

4.2.3 Part 3: Understanding the Pieces (45 min)

Project Structure Tour (15 min):

```

indicators/
  chapters/          # Your work goes here
    01-economy/
    02-transport/
    ...
  data/
    raw/             # Original data files
    processed/       # Cleaned data
    examples/        # Practice datasets
  scripts/
    R/               # Shared helper functions
  _quarto.yml        # Book configuration
  .git/              # Version control (hidden)

```

Code Chunk Anatomy (15 min):

```

#| label: my-chart      # Unique identifier
#| fig-cap: "Description" # Figure caption

library(tidyverse)      # Load tools
data <- read_csv("file.csv") # Import data
ggplot(data, aes(x, y)) + # Visualise
  geom_line()

```

Quarto Rendering Process (15 min):

- Markdown → formatted text
- Code chunks → executed by R → output
- Combined → HTML/PDF

Show the “Source” vs. “Visual” editor modes in Positron.

4.2.4 Part 4: Your First Modification (60 min)

Individual Exercise:

Each analyst modifies their example indicator:

1. Change the chart title to something descriptive
2. Change the line colour to "claret" (show `show_weca_palette()`)
3. Add a subtitle using the `subtitle` argument in `labs()`
4. Add a text section above the chart explaining what it shows
5. Re-render and verify changes

Stretch Goal (for faster learners):

- Add a second chart using `geom_point()` or `geom_col()`
- Filter the data to show only recent years using `filter(year >= 2020)`

Instructor: Circulate, answer questions, help with errors. Expect file path issues, typos, and R syntax confusion.

4.2.5 Part 5: Wrap-up & Homework (30 min)

Reflection (5 min):

Before we finish, take 2 minutes to write down:

1. One thing you understand now that you didn't at the start
2. One thing that's still unclear or you'd like more practise with

What surprised you about the code-first workflow? Share with the group if you're comfortable.

What We Learned Today:

- The complete workflow: data → code → chart → report
- How to run R code in Quarto
- How to render a chapter
- The WECA project structure

Looking Ahead:

- Session 2: Deep dive into R syntax and data manipulation
- Session 3: Cleaning messy data
- Session 4: Advanced Quarto features
- Session 5: Git collaboration

Homework (1-2 hours):

1. **Watch:** [Positron tidyverse demo](#)
2. **Practise:** Modify your example indicator chapter:
 - Add a second visualisation (bar chart or scatter plot)
 - Write 2-3 sentences describing what the data shows
 - Experiment with different WECA colours
3. **Explore:** Open `scripts/R/helpers.R` and read the function documentation
4. **Prepare:** Come with one question about something that confused you today

Optional Reading:

- [R for Data Science: Chapter 1 \(Introduction\)](#)
- [Quarto Guide: "Hello, Quarto" tutorial](#)

Chapter 5

Session 2: R Fundamentals

Master data transformation and visualisation

- **Duration:** 4 hours
- **Goal:** Gain confidence with R syntax, data transformation, and visualisation using the tidyverse.

In Session 1, you ran pre-written code. Today you learn to write your own – to ask questions of data and get visual answers. By the end, you’ll build an indicator chart from scratch.

5.1 Learning Outcomes

- Understand R’s key data structure (tibble/data frame)
- Use the pipe operator (`%>%` or `|>`) to chain operations
- Apply core dplyr verbs: `filter`, `select`, `mutate`, `summarise`, `group_by`
- Create multi-layered ggplot2 visualisations
- Debug common R errors

5.2 Session Structure

5.2.1 Part 1: Homework Review & Setup (30 min)

Homework Review (25 min):

- Show-and-tell: Volunteers demonstrate their homework modifications
- Address common questions from homework prep
- Quick poll: What was most confusing? (Adjust session focus accordingly)

Setting Up Your Practise Script (5 min):

Today you’ll write R code from scratch. You need somewhere to work:

1. In Positron, open the indicators project if it isn’t already open

2. File > New File > R Script (or Ctrl+Shift+N)
3. Save it straight away as `scripts/R/session-02-practice.R`
4. Add a header comment at the top:

```
# Session 2 Practice - R Fundamentals
# Your name, today's date
```

You'll use this file for all the “Try It” exercises today. Run individual lines with Ctrl+Enter or highlight a block and run it with Ctrl+Shift+Enter.

5.2.2 Part 2: R Basics - Data Structures (45 min)

Your indicator data arrives as a table of rows and columns – understanding tibbles is how R represents this.

The Tibble: R's Spreadsheet

```
library(tidyverse)

# Load a real WECA dataset
area_data <- read_csv(here::here("data", "examples", "area_employment.csv"))

# View it
area_data

# The $ operator accesses a single column (base R syntax)
area_data$employment_rate
```

Key Concepts:

- Tibbles are like Excel tables with named columns
- `<-` means “assign to” (store a value)
- `$` accesses a column
- R is case-sensitive: `Population` `population`
- Comments start with `#`

BREAK (15 min)

5.2.3 Part 3: The Tidyverse Way - Verbs and Pipes (60 min)

Building an indicator means filtering to your area, calculating rates, and summarising trends – these five verbs do exactly that.

The Pipe Operator:

```
# Old way (nested, hard to read)
summarise(filter(area_data, population > 100000),
           mean_pop = mean(population))

# Tidyverse way (sequential, readable)
area_data %>%
  filter(population > 100000) %>%
  summarise(mean_pop = mean(population))
```

Try It (5 min): Rewrite this nested expression using pipes:
`select(filter(area_data, employment_rate > 0.5), area,
employment_rate)`

The Five Core Verbs:

1. `filter()` - Keep rows that match a condition
2. `select()` - Keep specific columns
3. `mutate()` - Create new columns
4. `summarise()` - Calculate summary statistics
5. `group_by()` - Do calculations by category

Try It (25 min): Using your example dataset, work through each verb:

1. `filter()` – keep only rows for a specific area
2. `select()` – keep just the area, year, and value columns
3. `mutate()` – add a new column calculating a rate or percentage
4. `summarise()` – calculate the mean and max of a numeric column
5. `group_by()` %>% `summarise()` – calculate summary statistics by area

Pair up for these exercises. Person A writes the `filter()` and `select()` steps; Person B writes `mutate()` and `summarise()`. Swap roles for the `ggplot2` exercise in Part 4.

BREAK (15 min)

5.2.4 Part 4: ggplot2 - Grammar of Graphics (60 min)

Every indicator chapter needs at least one chart. This is how you build them.

```
# Layer 1: Data + aesthetic mapping
ggplot(bus_data, aes(x = year, y = ridership))

# Layer 2: Add geometry
ggplot(bus_data, aes(x = year, y = ridership)) +
  geom_line()

# Layer 3: Add styling
ggplot(bus_data, aes(x = year, y = ridership)) +
  geom_line(colour = get_weca_color("forest_green"), linewidth = 1) +
  geom_point(size = 2) +
  labs(title = "Bus Ridership Over Time",
       x = "Year",
       y = "Ridership (millions)") +
  theme_weca()
```

Key ggplot2 Geometries:

- `geom_line()` - Line charts (trends over time)
- `geom_point()` - Scatter plots (relationships)
- `geom_col()` - Bar charts (comparisons)

- `geom_smooth()` - Trend lines

Try It (10 min): Create a bar chart of employment rate by area using `geom_col()`. Add WECA colours and labels with `labs()`.

5.2.5 Part 5: Debugging R Errors (30 min)

Common Error Messages and Fixes:

1. **could not find function "filter"** → Load library: `library(tidyverse)`
2. **object 'data' not found** → Run the chunk that loads the data first
3. **unexpected symbol** → Cheque for missing commas, quotes, or parentheses
4. **'x' and 'y' lengths differ** → Data columns have different numbers of rows

Debugging Strategy:

1. Read the error message (bottom-up in stack trace)
2. Cheque the line number mentioned
3. Look for typos in variable/column names
4. Run code chunk-by-chunk to isolate the problem
5. Use `View(data)` to inspect your data frame, use the data explorer or use `data |> glimpse()`

5.2.6 Part 6: Wrap-up & Homework (15 min)

Reflection (5 min):

Before we finish, take 2 minutes to write down:

1. One thing you understand now that you didn't at the start
2. One thing that's still unclear or you'd like more practise with

Which dplyr verb felt most natural? Which was most confusing? Share with the group if you're comfortable.

Homework (1-2 hours):

1. **Complete:** The "R Basics" chapter in R for Data Science
2. **Practise:** Using your assigned example dataset:
 - Calculate 3 summary statistics (min, max, mean)
 - Create 2 different chart types
 - Add WECA theme and appropriate labels
3. **Create:** A new indicator section that includes data loading, transformation, visualisation, and findings paragraph
4. **Prepare:** Bring a real WECA dataset (CSV) for Session 3

Chapter 6

Session 3: Data Wrangling

Clean and prepare real-world data

- **Duration:** 4 hours
- **Goal:** Master the skills to clean, reshape, and prepare messy real-world data for analysis.

Real data is messy. Column names are inconsistent, values are missing, formats are wrong. Data wrangling is where analysts spend most of their time – and it’s the step that’s invisible in Excel. Today you learn to do it reproducibly.

6.1 Learning Outcomes

- Import data from CSV and Excel files
- Handle missing values and data quality issues
- Reshape data between wide and long formats
- Join multiple datasets together
- Apply data cleaning best practises
- Use the `here()` package for robust file paths

6.2 Session Structure

6.2.1 Part 1: Homework Review & Setup (30 min)

Homework Review (25 min):

- Volunteer demos of homework indicators
- Quick discussion: What datasets did you bring? What challenges do you anticipate?

Setting Up Your Practise Script (5 min):

As in Session 2, create a new R script for today’s exercises:

1. In Positron, File > New File > R Script (or Ctrl+Shift+N)
2. Save as `scripts/R/session-03-practice.R`
3. Add a header and load the libraries you'll need:

```
# Session 3 Practice - Data Wrangling
# Your name, today's date

library(tidyverse)
library(readxl)
library(janitor)
source(here::here("scripts", "R", "helpers.R"))
```

Run lines with **Ctrl+Enter** or highlight a block and use **Ctrl+Shift+Enter**. Keep your code as you go – this file becomes a useful reference for your own indicator work.

6.2.2 Part 2: Importing Data (45 min)

Your indicator starts with a CSV or Excel download. Getting it into R cleanly is the first step.

Reading CSV Files:

```
library(tidyverse)
library(readxl)
source(here::here("scripts", "R", "helpers.R"))

# The WECA way
data <- load_csv(here::here("data", "raw", "indicator_data.csv"))
```

Reading Excel Files:

```
# Basic read
data <- read_excel("data/raw/employment_data.xlsx")

# Specific sheet
data <- read_excel("data/raw/employment_data.xlsx", sheet = "Q4_2024")

# Skip header rows
data <- read_excel("data/raw/employment_data.xlsx", sheet = "Data", skip = 5)
```

File Path Best Practises:

```
# Good - relative paths with here()
data <- load_csv(here::here("data", "raw", "myfile.csv"))

# Bad - absolute paths (breaks on other computers)
data <- read_csv("C:/Users/steve/projects/indicators/data/raw/myfile.csv")
```

Try It (10 min): Load your own CSV with `read_csv()`, then inspect it with `glimpse()`. How many rows and columns does it have? What types are the columns?

BREAK (15 min)

6.2.3 Part 3: Data Cleaning Essentials (60 min)

Real government data has messy names, missing values, and mixed types. This is how you fix them.

Cleaning Column Names:

```
library(janitor)

data_clean <- data %>%
  clean_names() # Converts "Area Name" to "area_name"
```

Try It (5 min): Run `clean_names()` on your dataset. Compare `names(data)` before and after.

Handling Missing Values:

```
# Check for missing data
check_missing(data)

# Remove rows with missing values
data_complete <- data %>%
  drop_na(year, value)

# Replace missing values
data_filled <- data %>%
  mutate(
    value = if_else(is.na(value), 0, value),
    category = replace_na(category, "Unknown")
  )
```

Try It (10 min): Cheque your data for NAs with `check_missing()`. Decide how to handle them – drop the rows, fill with zeros, or replace with “Unknown”?

Type Conversions:

```
data <- data %>%
  mutate(
    year = as.numeric(year),
    date = as.Date(date, format = "%d/%m/%Y"),
    region = factor(region, levels = c("Bristol", "Bath", "South Glos", "North Somerset"))
  )
```

BREAK (15 min)

6.2.4 Part 4: Reshaping Data (60 min)

ONS data often arrives in wide format (one column per year). Your charts need long format.

Understanding Wide vs. Long Format:

Wide format (like Excel):

```
year   bristol_value  bath_value  sg_value
```

2020	1000	500	750
------	------	-----	-----

Long format (tidy):

year	area	value
2020	Bristol	1000
2020	Bath	500
2020	South Glos	750

Pivot Longer - Wide to Long:

```
data_long <- data_wide %>%
  pivot_longer(
    cols = bristol_value:sg_value,
    names_to = "area",
    values_to = "value"
  ) %>%
  mutate(
    area = str_remove(area, "_value"),
    area = str_to_title(area)
  )
```

Try It (10 min): Reshape a wide example dataset to long format using `pivot_longer()`. Cheque the result with `glimpse()`.

BREAK (15 min)

6.2.5 Part 5: Joining Datasets (30 min)

Many indicators combine data from multiple sources – e.g. population denominators with raw counts.

Types of Joins:

```
# Left join - keep all rows from first table
combined <- data1 %>%
  left_join(data2, by = "area")

# Join on multiple columns
combined <- data1 %>%
  left_join(data2, by = c("area", "year"))
```

6.2.6 Common Pitfalls

1. **`pivot_longer()` selects wrong columns** – Use `cols = starts_with("x")` or name columns explicitly. Cheque with `names(data)` first.
2. **Join produces more rows than expected** – Duplicate keys in one table cause row multiplication. Cheque with `count(data, key_column)`.
3. **`read_excel()` reads numbers as text** – Excel formatting can trick R. Use `mutate(value = as.numeric(value))` and cheque for introduced NAs.
4. **File not found with `here()`** – Ensure you're in the R Project. Run `here::here()` alone to cheque the project root.

6.2.7 Part 6: Real Data Workshop (45 min)

Before coding, write down 2-3 specific questions your indicator data could answer. Pick one to focus on. What chart type would best communicate the answer?

Work in pairs. Each pair takes one dataset. Divide the work: one person handles import and cleaning, the other handles reshaping and visualisation. Review each other's code before combining.

Work with real WECA datasets:

1. Import the data
2. Inspect and document issues
3. Clean column names
4. Handle missing values
5. Reshape if needed
6. Create a visualisation
7. Calculate summary statistics

6.2.8 Part 7: Wrap-up & Homework (15 min)

Reflection (5 min):

Before we finish, take 2 minutes to write down:

1. One thing you understand now that you didn't at the start
2. One thing that's still unclear or you'd like more practise with

What was the messiest thing about your data, and how did you fix it? Share with the group if you're comfortable.

Homework (1-2 hours):

1. **Complete:** Prepare your real indicator data (cleaned version in `data/processed/`)
2. **Create:** A draft indicator section with data loading, cleaning, transformation, chart, and findings
3. **Read:** CONTRIBUTING.md sections on "Code Style Guide"

Chapter 7

Session 4: Quarto Mastery

Author professional reproducible reports

- **Duration:** 4 hours
- **Goal:** Author professional, reproducible reports that combine narrative, code, visualisations, and tables.

So far you’ve written code that produces charts. Today you turn that into a professional document – one that updates itself when data changes, never has a copy-paste error, and can produce HTML and PDF from the same source.

7.1 Learning Outcomes

- Structure multi-section Quarto documents
- Control code chunk execution and output
- Format text with Markdown syntax
- Create professional tables
- Use cross-references for figures and tables
- Understand the “freeze” workflow for collaboration
- Render both HTML and PDF outputs

7.2 Session Structure

7.2.1 Part 1: Homework Review (30 min)

- Show-and-tell: Volunteers share their cleaned datasets and draft indicators
- Discussion: What data cleaning challenges did you encounter?

7.2.2 Part 2: Markdown Essentials (45 min)

Your indicator chapter needs headings, formatted text, and structured findings – not just code.

Basic Syntax:

```
# Heading 1
## Heading 2
### Heading 3

**Bold text**
*Italic text*

- Bullet point
- Another point
  - Nested point

1. Numbered list
2. Second item

[Link text](https://example.com)

> Block quote - for highlighting key findings

`inline code` for variable names like `employment_rate`
```

Guided Practise (20 min):

Add section headings, format key numbers, create bulleted lists, and add block quotes to your chapter.

7.2.3 Part 3: Quarto Code Chunks (60 min)

Control what readers see: hide the data wrangling, show the chart, embed key numbers in your narrative.

Anatomy of a Code Chunk:

```
```{r}
#| label: my-analysis
#| echo: false
#| warning: false
#| message: false
#| fig-cap: "Employment trends across the region"
#| fig-width: 8
#| fig-height: 5

Your R code here
ggplot(data, aes(x = year, y = value)) +
 geom_line() +
 theme_weca()
```
```

Key Chunk Options:

| Option | Effect | When to Use |
|------------------------------|---------------------------|--------------------------------|
| <code>echo: false</code> | Hide code, show output | Final reports |
| <code>echo: true</code> | Show code and output | Teaching, documentation |
| <code>eval: false</code> | Show code, don't run | Example code |
| <code>include: false</code> | Run code, hide everything | Setup chunks |
| <code>warning: false</code> | Suppress warnings | Known harmless warnings |
| <code>fig-cap: "..."</code> | Figure caption | All charts (for accessibility) |
| <code>label: my-chunk</code> | Chunk identifier | Required for cross-references |

Try It (10 min): Add `echo: false` and `fig-cap` to your existing chart chunk, then re-render. Notice the difference in the output.

BREAK (15 min)

Inline R Code:

Use inline R expressions to embed computed values directly in your narrative text.

Example syntax:

In your `.qmd` file you would write:

```
The employment rate increased from `r min(data$employment_rate)`%
in 2015 to `r max(data$employment_rate)`% in 2024.
```

How it works:

- Wrap R expressions with backtick-r-space: ``r expr``
- The code executes when rendering
- Results appear inline in your text

Benefits:

- Numbers update automatically when data changes
- No copy-paste errors
- Ensures text matches visualisations

Try It (10 min): Write a sentence with an inline R expression for a summary statistic from your indicator data (e.g. the mean, min, or max of a key column).

BREAK (15 min)

7.2.4 Part 4: Tables in Quarto (45 min)

Summary tables alongside charts make your indicator section complete.

Creating Tables from Data:

```
# Basic table
data %>%
  select(area, population, employment_rate) %>%
  knitr::kable(
    col.names = c("Area", "Population", "Employment Rate (%)" ),
    digits = 1,
```

```
format.args = list(big.mark = ",")
)
```

Try It (10 min): Create a `knitr::kable()` table from your indicator data, with formatted column names and appropriate decimal places.

7.2.5 Part 5: Document Structure & Cross-References (45 min)

Professional reports reference “see Figure 1” not “see the chart above”.

YAML Frontmatter:

```
---
title: "Public Transport Connectivity"
subtitle: "Connecting the region through better public transport"
author: "Your Name"
date: last-modified
---
```

Cross-References to Figures:

```
```{r}
#| label: fig-employment-trend
#| fig-cap: "Employment rate trends 2015-2024"

ggplot(data, aes(x = year, y = employment_rate)) +
 geom_line() +
 theme_weca()
```
```

As shown in @fig-employment-trend, employment rates have steadily increased over the past decade.

Try It (10 min): Add a label to your figure chunk (e.g. `#| label: fig-my-chart`) and reference it in your text with `@fig-my-chart`. Re-render and cheque the link works.

7.2.6 Common Pitfalls

1. **Cross-reference doesn't link** – Label must start with `fig-` or `tbl-` prefix. Cheque: `#| label: fig-my-chart` not `#| label: my-chart`.
2. **YAML error on render** – Indentation matters. Use 2 spaces, never tabs. Cheque colons have a space after them.
3. **Inline R shows code instead of result** – Ensure backtick-r-space syntax is correct and the referenced object exists in an earlier chunk.
4. **Figure caption missing from output** – Need both `#| label:` and `#| fig-cap:` in the same chunk.

7.2.7 Part 6: The Freeze Workflow for Collaboration (30 min)

How It Works:

1. You render your chapter with your local data
2. Quarto saves outputs to `_freeze/` directory
3. You commit both source (`.qmd`) and cache (`_freeze/`) to Git
4. Others pull your changes and render the full book using your cached outputs

Workflow in Practise:

```
# After editing your chapter
quarto render chapters/02-transport/index.qmd

# Commit BOTH source and cache
git add chapters/02-transport/index.qmd
git add _freeze/html/chapters/02-transport/
git commit -m "Add bus ridership indicator"
git push
```

Important: Always commit the `_freeze/` directory alongside your `.qmd` file!

7.2.8 Part 7: Rendering Options (15 min)

```
# HTML (default)
quarto render chapters/02-transport/index.qmd

# PDF
quarto render chapters/02-transport/index.qmd --to pdf

# Preview with live reload
quarto preview
```

7.2.9 Part 8: Practical Exercise (45 min)

Write a “Key Question” for your indicator section. Your analysis should answer this question. Your Key Findings section should directly address it.

Work in pairs. Review each other’s Quarto source for: missing figure captions, chunk labels, cross-references, and narrative quality.

Create a complete indicator section with:

1. YAML frontmatter with title, author, date
2. Context section explaining the indicator
3. Data section with code folding
4. Analysis section with visualisations, tables, and cross-references
5. Key findings section with bulleted insights
6. Data source citation

7.2.10 Part 9: Wrap-up & Homework (15 min)

Reflection (5 min):

Before we finish, take 2 minutes to write down:

1. One thing you understand now that you didn't at the start
2. One thing that's still unclear or you'd like more practise with

What's the biggest advantage of Quarto over copy-paste reporting? Share with the group if you're comfortable.

Homework (1-2 hours):

1. **Complete:** Your indicator section following the structure
2. **Create:** A second indicator using a different dataset
3. **Refine:** Ensure all charts have captions, add inline R code to narrative
4. **Read:** CONTRIBUTING.md sections on "Collaborative Rendering"

Chapter 8

Session 5: Git Collaboration

Version control and code review

- **Duration:** 4 hours
- **Goal:** Use Git and GitHub for version control, collaboration, and code review in the WECA indicators project.

8.1 Learning Outcomes

- Understand version control concepts (commits, branches, merges)
- Use Git Bash for basic version control operations
- Create and switch between branches
- Stage, commit, and push changes
- Create and review pull requests on GitHub
- Resolve basic merge conflicts
- Follow the WECA project Git workflow

8.2 Session Structure

8.2.1 Part 1: Homework Review & Version Control Intro (45 min)

Show-and-tell (20 min): Volunteers present completed indicator sections.

Why Version Control? (25 min)

The Problem:

```
indicator_analysis_v1.qmd
indicator_analysis_v2.qmd
indicator_analysis_v2_final.qmd
indicator_analysis_v2_final_ACTUAL.qmd
```

Git Concepts:

| Concept | Analogy | Purpose |
|------------------------|----------------------------------|----------------------------------|
| Repository | Project folder + time machine | Contains all files + history |
| Commit | Snapshot/checkpoint | Save point you can return to |
| Branch | Parallel timeline | Work without affecting main |
| Merge | Combine timelines | Integrate your work back to main |
| Remote (GitHub) | Cloud backup + collaboration hub | Share work, collaborate |

8.2.2 Part 2: Git Basics in Git Bash (60 min)

Essential Commands:

1. Cheque Status:

```
git status
```

2. View Changes:

```
git diff chapters/02-transport/index.qmd
```

3. Stage Changes:

```
git add chapters/02-transport/index.qmd
git add _freeze/html/chapters/02-transport/
```

4. Commit:

```
git commit -m "Add bus ridership indicator to transport chapter"
git log --oneline
```

5. Push:

```
git push
```

BREAK (15 min)

Guided Practise (30 min):

1. Open Git Bash in the indicators project
2. Make a small change to your chapter
3. Cheque status: `git status`
4. View changes: `git diff`
5. Stage: `git add chapters/XX-topic/index.qmd`
6. Commit: `git commit -m "Update chapter introduction"`
7. View history: `git log --oneline`

8.2.3 Part 3: Branching Workflow (60 min)

Why Branches?

- Work on your indicator without affecting others
- Experiment safely
- Required for pull request workflow

Branch Naming Convention:

yourname/chapter-or-feature

Examples:

heather/transport

alex/environment-update

sarah/fix-chart-colours

Creating and Using Branches:

```
# See current branch
git branch

# Create and switch to new branch
git checkout -b yourname/transport

# Make changes, stage, and commit
git add chapters/02-transport/index.qmd
git commit -m "Add bus ridership indicator"

# Push branch to GitHub
git push -u origin yourname/transport

# Switch branches
git checkout main
git checkout yourname/transport
```

BREAK (15 min)

Guided Practise (30 min):

1. Create a new branch: `git checkout -b firstname/chapter-name`
2. Make a change to your chapter
3. Stage and commit
4. Push: `git push -u origin firstname/chapter-name`
5. Switch to main: `git checkout main`
6. Switch back: `git checkout firstname/chapter-name`

8.2.4 Part 4: Pull Requests on GitHub (60 min)

What is a Pull Request?

A formal request to merge your branch into main. Enables:

- Code review before merging
- Discussion and feedback

- Quality control
- Documentation of changes

Creating a Pull Request:

Step 1: Push your branch (already done)

Step 2: Go to GitHub and create PR

Step 3: Fill in PR template with:

- Title: “Add bus ridership indicator to transport chapter”
- Summary of changes
- Data sources
- Checklist (code runs, charts labelled, etc.)

Step 4: Request review

Step 5: Submit

BREAK (15 min)

Reviewing a Pull Request:

1. Go to “Pull Requests” tab on GitHub
2. Click on a PR
3. Click “Files changed” tab
4. Review code quality, chart labels, documentation
5. Leave comments
6. Submit review: Approve, Request changes, or Comment

Guided Practise (40 min):

Part A: Each analyst creates a PR for their branch (20 min)

Part B: Peer review in pairs (20 min)

8.2.5 Part 5: Merging and Updating (30 min)

Merging a Pull Request:

1. Click “Merge pull request” on GitHub
2. Confirm the merge
3. Delete the branch

Locally, update your main:

```
# Switch to main
git checkout main

# Pull latest changes
git pull

# Delete local feature branch
git branch -d yourname/chapter-name
```

8.2.6 Part 6: Merge Conflicts (Basic) (30 min)

What is a Merge Conflict?

Occurs when two people edit the same lines of the same file.

Conflict Markers:

```
<<<<<< HEAD
Bus ridership increased by 15% over the period.
=====
Bus ridership showed steady growth, increasing by 12% overall.
>>>>>> yourname/transport
```

Resolution Process:

1. Identify the conflict
2. Open the file in Positron
3. Review both versions
4. Choose or combine the changes
5. Remove conflict markers
6. Save, stage, and commit

8.2.7 Part 7: WECA Project Git Workflow (20 min)

Standard Workflow:

```
# 1. Start from main, get latest
git checkout main
git pull

# 2. Create feature branch
git checkout -b yourname/feature-name

# 3. Do your work

# 4. Stage and commit
git add file1.qmd
git commit -m "Clear, descriptive message"

# 5. Push branch
git push -u origin yourname/feature-name

# 6. Create Pull Request on GitHub

# 7. After merge, update main
git checkout main
git pull
git branch -d yourname/feature-name
```

Commit Message Best Practises:

Good: “Add bus ridership indicator to transport chapter” Bad: “Updates”, “Fixed stuff”, “WIP”

8.2.8 Part 8: Final Practise - Full Workflow (45 min)

Complete the full Git workflow end-to-end:

1. Update main branch
2. Create a new branch for a new indicator
3. Add content to your chapter
4. Commit changes (remember freeze files!)
5. Push branch
6. Create Pull Request on GitHub
7. Review a peer's PR

8.2.9 Part 9: Course Wrap-up (30 min)

Reflection (5 min):

Before we finish, take 2 minutes to write down:

1. One thing you understand now that you didn't at the start
2. One thing that's still unclear or you'd like more practise with

What part of the Git workflow would you want to practise more? Share with the group if you're comfortable.

What We've Accomplished:

Over 5 sessions, you've learned to:

- Navigate the command line with Git Bash
- Use Positron IDE for R development
- Write R code using tidyverse
- Create visualisations with ggplot2
- Author reproducible reports with Quarto
- Use Git and GitHub for collaboration

Next Steps:

1. Continue building indicators for your assigned priority areas
2. Ask for help in team chat or via PR reviews
3. Reference documentation
4. Practise regularly

Feedback Session (15 min):

- What worked well?
- What was most challenging?
- What additional support would be helpful?

Q&A (15 min)

Part III

Resources

Chapter 9

Practice Datasets

9.1 Example Datasets

Two small, clean datasets are provided for practice exercises in Sessions 1-2. From Session 3 onwards, analysts work with their own real indicator data.

9.1.1 Bus Ridership

File: bus_ridership.csv

Description: Annual bus ridership data for the WECA region

Columns:

- year - Year (2015-2024)
- ridership - Annual ridership in millions
- routes - Number of bus routes
- region - Geographic area

Use cases:

- Session 1: Creating first visualisation
- Session 2: Line charts and trends with ggplot2

9.1.2 Area Employment

File: area_employment.csv

Description: Employment rate statistics by WECA local authority (2019-2023)

Columns:

- area - Local authority name (Bath and North East Somerset, Bristol, North Somerset, South Gloucestershire)
- year - Year (2019-2023)
- employment_rate - Proportion employed, 16-64 age group (0-1 scale)
- population - Total population

Use cases:

- Session 2: Filtering, grouping, summarising with dplyr verbs
- Session 2: Bar charts and comparisons with ggplot2

9.2 Real Indicator Data

From Session 3 onwards, analysts bring their own real WECA datasets. These will typically be messier than the example data above – see Data Cleaning Notes below for common issues and solutions.

9.3 Data Cleaning Notes

Common issues to watch for:

1. **Column naming:**
 - Mixed case: “Area Name”, “area_name”, “AREA_NAME”
 - Special characters: “Year (2023)”, “Percentage (%)”
 - Solution: Use `janitor::clean_names()`
2. **Missing values:**
 - Coded as: “N/A”, “n/a”, “-”, “*”, empty cells
 - Solution: Explicit `is.na()` checks and `replace_na()`
3. **Date formats:**
 - “31/12/2023”, “2023-12-31”, “December 2023”
 - Solution: Use `as.Date()` with appropriate format strings
4. **Numeric values as text:**
 - “1,234”, “£50.00”, “12.5%”
 - Solution: `str_remove_all()` non-numeric characters, then `as.numeric()`
5. **Excel header rows:**
 - Title rows, merged cells, footnotes
 - Solution: Use `skip` = parameter in `read_excel()`

9.4 Dataset Documentation Template

When adding new practice datasets, document them using this template:

```
### Dataset Name

**File:** `filename.csv`

**Description:** Brief description of what the data contains

**Columns:**

- `column1` - Description
- `column2` - Description

**Source:** Data source and URL

**Date range:** YYYY-YYYY
```

****Use cases:**** Which sessions/exercises use this data

****Known issues:**** Any data quality notes

Chapter 10

Setup Guide

10.1 Pre-Session Setup Checklist

Complete this setup **before Session 1** to ensure smooth training delivery.

10.2 Software Installation

10.2.1 Required Software

Install the following on all analyst machines:

1. **R** (version 4.3)
 - Download: <https://cran.r-project.org/>
 - Windows: Use default installation options
 - Verify: Open R console, check version with `R.version.string`
2. **Positron** (Recommended IDE)
 - Download: <https://github.com/posit-dev/positron/releases>
 - Alternative: RStudio if Positron unavailable
 - Verify: Launch Positron, create new R file
3. **Quarto** (version 1.4)
 - Download: <https://quarto.org/docs/get-started/>
 - Verify: Open terminal, run `quarto --version`
4. **Python** (version 3.10)
 - Download: <https://www.python.org/>
 - Windows: Check “Add Python to PATH” during installation
 - Verify: `python --version`
5. **uv** (Python package manager)
 - Install: `pip install uv` or download from <https://github.com/astral-sh/uv>
 - Verify: `uv --version`
6. **Git** (version control)
 - Download: <https://git-scm.com/>
 - Windows: Install Git Bash with default options
 - Verify: Open Git Bash, run `git --version`

10.3 Project Setup

10.3.1 Clone the Indicators Repository

```
cd ~/projects
git clone https://github.com/your-org/indicators.git
```

10.3.2 Install R Packages

1. Open Positron
2. Navigate to: File > Open Folder > indicators
3. In R console, run:

```
# Install renv (if not already installed)
install.packages("renv")

# Restore project packages
renv::restore()
```

This will install all required R packages listed in `renv.lock`.

Expected packages include:

- tidyverse (data manipulation and visualisation)
- here (robust file paths)
- readxl (Excel file import)
- janitor (data cleaning)
- gt (publication tables)
- And other project-specific packages

10.3.3 Install Python Dependencies

1. Open Git Bash in the indicators project folder
2. Run:

```
# Create virtual environment and install packages
uv sync
```

This installs Python packages listed in `pyproject.toml`.

10.3.4 Install Pre-Commit Hooks

Run the hook installation script:

```
cd indicators
bash scripts/hooks/install-hooks.sh
```

This sets up secret scanning to prevent accidental credential commits.

10.4 Verify Setup

10.4.1 Check R Environment

In Positron R console:

```
# Load core packages
library(tidyverse)
library(here)

# Check project root
here::here() # Should show indicators project path

# Check WECA helpers
source(here::here("scripts", "R", "helpers.R"))
show_weca_palette()
```

10.4.2 Check Quarto Rendering

```
cd indicators
quarto render index.qmd
```

Should produce HTML output in `_output/` directory without errors.

10.4.3 Check Git Configuration

```
git config user.name
git config user.email
```

If not set, configure:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

10.4.4 Test Full Workflow

1. Open `chapters/01-economy/index.qmd` in Positron
2. Run the first code chunk (Ctrl+Shift+Enter)
3. Render the chapter: Click “Render” button
4. Verify HTML output opens in browser

10.5 Prepare Practice Data

10.5.1 Example Datasets

Ensure these files exist in `indicators/data/examples/`:

- `bus_ridership.csv`
- `area_employment.csv`

These are small, clean datasets (4 columns, 20 rows) used for practice exercises in Sessions 1-2. From Session 3 onwards, analysts work with their own real indicator data.

10.5.2 Create Practice Branches

For each analyst, create a practice branch:

```
git checkout -b helen-practice
git push -u origin helen-practice
git checkout main

git checkout -b alex-practice
git push -u origin alex-practice
git checkout main

# Repeat for each analyst
```

10.6 Troubleshooting Setup

If you encounter problems during setup, see the [Troubleshooting Guide](#) — particularly the **Setup Issues** section.

10.7 Post-Setup Verification

Before Session 1, verify each machine:

- ☐ R opens and shows correct version
- ☐ Positron launches without errors
- ☐ Indicators project opens in Positron
- ☐ `library(tidyverse)` loads without errors
- ☐ `here::here()` shows correct project path
- ☐ Test chapter renders to HTML
- ☐ Git Bash opens and shows correct git version
- ☐ `git status` works in indicators folder
- ☐ Example datasets are present in `data/examples/`

10.8 Support Resources

During setup:

- Installation guides: Check software websites
- IT support: Contact your IT department
- Course instructor: Email for specific issues

During course:

- R help: `?function_name` or `help(package_name)`
- Tidyverse docs: <https://www.tidyverse.org/>
- Quarto guide: <https://quarto.org/docs/guide/>

- Git reference: <https://git-scm.com/doc>

Chapter 11

Troubleshooting Guide

11.1 Setup Issues

These problems typically occur during initial installation and project setup.

11.1.1 “Quarto not found”

Symptom: `quarto: command not found`

Solution:

- Restart terminal/Positron after Quarto installation
- Check PATH includes Quarto installation directory
- Windows: May need to add to PATH manually

11.1.2 “Python package installation fails”

Symptom: `uv sync` errors

Solution:

```
# Update uv
pip install --upgrade uv

# Clear cache and retry
uv cache clear
uv sync
```

11.1.3 “renv restore hangs”

Symptom: Package restoration taking very long or hanging

Solution:

- Check internet connection
- Try individual package installation
- Clear renv cache: `renv::purge()`

11.2 Common Issues and Solutions

11.2.1 R Issues

11.2.1.1 “Package not found”

Error message:

Error in library(tidyverse): there is no package called 'tidyverse'

Solution:

```
# Restore all project packages
renv::restore()

# Or install individually
install.packages("tidyverse")
```

Prevention: Run `renv::restore()` after cloning the repository.

11.2.1.2 “File not found”

Error message:

Error: 'data/myfile.csv' does not exist

Cause: Hardcoded paths or wrong working directory

Solution:

```
# Always use here::here() for file paths
data <- read_csv(here::here("data", "raw", "myfile.csv"))

# Check your project root
here::here() # Should show indicators project directory
```

Prevention: Never use absolute paths like `C:/Users/name/...`

11.2.1.3 “Object not found”

Error message:

Error: object 'my_data' not found

Cause: Code chunks run out of order

Solution:

1. Restart R session: `Session > Restart R` in Positron
2. Run code chunks sequentially from top to bottom
3. Check for typos in variable names (R is case-sensitive)

11.2.1.4 Rendering errors

Error message:

Error in eval(expr, envir, enclos): could not find function...

Solution:

1. Check R console for specific error message
2. Ensure all required packages are loaded
3. Verify all code chunks before the error run successfully
4. Try rendering in a clean R session

Debug strategy:

```
# Restart R session
.rs.restartR()

# Run chunks one-by-one until you find the problematic chunk
```

11.2.2 Git Issues

11.2.2.1 “Permission denied (publickey)”

Error message:

```
Permission denied (publickey).
fatal: Could not read from remote repository.
```

Cause: SSH keys not configured

Solution: Use HTTPS instead of SSH

```
# Check current remote URL
git remote -v

# If using SSH (git@github.com:...), switch to HTTPS
git remote set-url origin https://github.com/your-org/indicators.git
```

11.2.2.2 “Merge conflict”

Symptom: Git refuses to merge, shows conflict markers in file

What it looks like:

```
<<<<<<< HEAD
Your changes
=====
Someone else's changes
>>>>>>> branch-name
```

Solution:

1. Open the file in Positron
2. Find the conflict markers (<<<<<<<, =====, >>>>>>>)
3. Decide which version to keep (or combine both)
4. Delete the conflict markers
5. Save the file
6. Stage and commit:

```
git add filename.qmd
git commit -m "Resolve merge conflict"
```

Prevention:

- Pull from main before creating new branches
- Communicate with team about who's working on what
- Work on different sections when possible

11.2.2.3 “Your branch is behind origin/main”**Error message:**

Your branch is behind 'origin/main' by 3 commits

Solution:

```
# Pull latest changes before continuing work
git pull

# If you have local commits, this will create a merge commit
# Or use rebase for cleaner history (advanced)
git pull --rebase
```

11.2.2.4 Can't push (rejected)**Error message:**

! [rejected] main -> main (fetch first)

Cause: Remote has commits you don't have locally

Solution:

```
# Pull first, then push
git pull
git push
```

11.2.3 Quarto Issues**11.2.3.1 “Kernel error”****Error message:**

Kernel error: Cannot start kernel

Cause: R/Python kernel not found by Quarto

Solution:

1. Verify R/Python installation
2. Restart Positron
3. Check Quarto can find R:

```
quarto check
```

11.2.3.2 “LaTeX error” (PDF rendering)**Error message:**

! LaTeX Error: File 'xyz.sty' not found

Cause: Missing LaTeX installation for PDF output

Solution:

```
# Install TinyTeX
quarto install tinytex
```

Alternative: Focus on HTML output (PDF is optional)

11.2.3.3 Freeze cache issues

Symptom: Changes to code don't appear in rendered output

Solution:

```
# Force re-render of chapter
quarto render chapters/XX-topic/index.qmd --execute-freeze refresh

# Or delete freeze cache
rm -rf _freeze/html/chapters/XX-topic/
quarto render chapters/XX-topic/index.qmd
```

When cache conflicts in merge:

1. Delete local `_freeze/` for that chapter
2. Pull latest from GitHub
3. Re-render the chapter

11.2.4 Positron Issues

11.2.4.1 Code chunks don't execute

Symptom: Pressing Ctrl+Shift+Enter does nothing

Solution:

1. Check R session is active (look for R console at bottom)
2. Try **Session > Restart R**
3. Verify renv is activated (session info should show renv)
4. Close and reopen the `.qmd` file

11.2.4.2 File doesn't open

Symptom: File shows as binary or doesn't open properly

Solution:

1. Check file path is correct (use tab completion)
2. Try **File > Open Project** and reopening project
3. Check file encoding (should be UTF-8)

11.2.5 Data Issues

11.2.5.1 CSV encoding problems

Symptom: Strange characters (Ã©, Â£, etc.) in imported data

Solution:

```
# Specify encoding explicitly
data <- read_csv("file.csv", locale = locale(encoding = "Latin1"))
# or
data <- read_csv("file.csv", locale = locale(encoding = "UTF-8"))
```

11.2.5.2 Excel sheets not importing

Symptom: read_excel() fails or imports wrong data

Solution:

```
# Check available sheet names
excel_sheets("file.xlsx")

# Read specific sheet
data <- read_excel("file.xlsx", sheet = "Sheet1")

# Skip header rows
data <- read_excel("file.xlsx", sheet = "Data", skip = 5)

# Specify range
data <- read_excel("file.xlsx", sheet = "Data", range = "A5:D50")
```

11.2.5.3 Type conversion errors

Symptom: Numbers treated as text, dates not parsing

Solution:

```
# Check column types
glimpse(data)

# Convert text to numeric
data <- data %>%
  mutate(
    year = as.numeric(year),
    value = as.numeric(value)
  )

# Convert to dates
data <- data %>%
  mutate(
    date = as.Date(date, format = "%d/%m/%Y")
  )
```

11.2.6 Performance Issues

11.2.6.1 Rendering very slow

Symptom: quarto render takes minutes

Cause: Large datasets or complex computations

Solution:

1. Use `freeze: auto` (already configured)
2. Cache expensive operations:

```
# Save processed data
write_rds(processed_data, here::here("data", "processed", "cached_data.rds"))

# Load cached version
processed_data <- read_rds(here::here("data", "processed", "cached_data.rds"))
```

3. Sample data during development:

```
# Use smaller sample during development
data_sample <- data %>% slice_sample(n = 1000)
```

11.2.6.2 R session crashes

Symptom: Positron becomes unresponsive

Cause: Memory issues with large datasets

Solution:

1. Restart R session frequently
2. Remove large objects when done:

```
rm(large_object)
gc() # Garbage collection
```

3. Work with smaller data subsets during development

11.3 Getting Help

11.3.1 Self-Help Resources

1. **R documentation:**

```
?function_name
help(package = "packagename")
```

2. **Package vignettes:**

```
vignette("dplyr")
browseVignettes("ggplot2")
```

3. **Online resources:**

- [R for Data Science](#)

- [Quarto documentation](#)
- [Git documentation](#)

11.3.2 Asking for Help

When posting questions to team chat or asking instructor:

Good question format:

Problem: When I try to render my chapter, I get error X

What I tried:

- Restarted R session
- Ran `renv::restore()`
- Checked the code in the chunk before error

Error message:

[paste full error message]

Reproducible example:

[paste minimal code that reproduces error]

Information to include:

- What you were trying to do
- What you expected to happen
- What actually happened (including full error message)
- What you've already tried
- Your R version (`R.version.string`)
- Package versions (`packageVersion("package_name")`)

11.3.3 Office Hours

Post-course office hours schedule:

- **When:** Wednesdays, 2-3pm
- **Where:** Team meeting room / Video call
- **Book:** Email instructor or post in team chat

11.3.4 Team Support

- **Team chat:** #indicators-project channel
- **Peer support:** Pair programming sessions
- **Code review:** PR comments and discussions